Multiple regression

Lecture notes

Foundations of Regression and Predictive Modeling

Table of contents

From one input to several . . . 1
The AI prompts example . . . 2
Adjusting for pull request size . . . 4
Adjustment as subtraction . . . 6
Overall versus partial . . . 7
A pricing question . . . 8
Marketing confounding . . . 9
Exposure adjustment . . . 10
Market adjustment . . . 11
Revenue implication . . . 13
Interactions . . . 13
Reading the interaction . . . 14
Fixed effects as many dummies . . . 15
What to carry forward . . . 16

From one input to several

Simple regression asks:

$$\text{How does } y \text{ vary with } x?$$

Multiple regression asks:

$$\text{How does } y \text{ vary with one } x \text{ after adjusting for another } x?$$

That phrase, after adjusting, is the key move. A coefficient in multiple regression is a partial slope. It describes a comparison holding the other variables in the model fixed.

The two-predictor model is

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + e_i.$$

The fitted value is

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2}.$$

The AI prompts example

In the SDS capstone example, each row is a merged pull request. The variables are:

- `HoursToMerge`: hours from opening the pull request to merge.
- `AIPrompts`: number of AI-assistant prompts linked to the pull request.
- `LinesChanged`: total lines added plus deleted.

Read the data:

```
prs <- read_csv(file.path(module_dir, "ai_pull_requests.csv")) |>
  mutate(Lines100 = LinesChanged / 100)
```

The naive model is

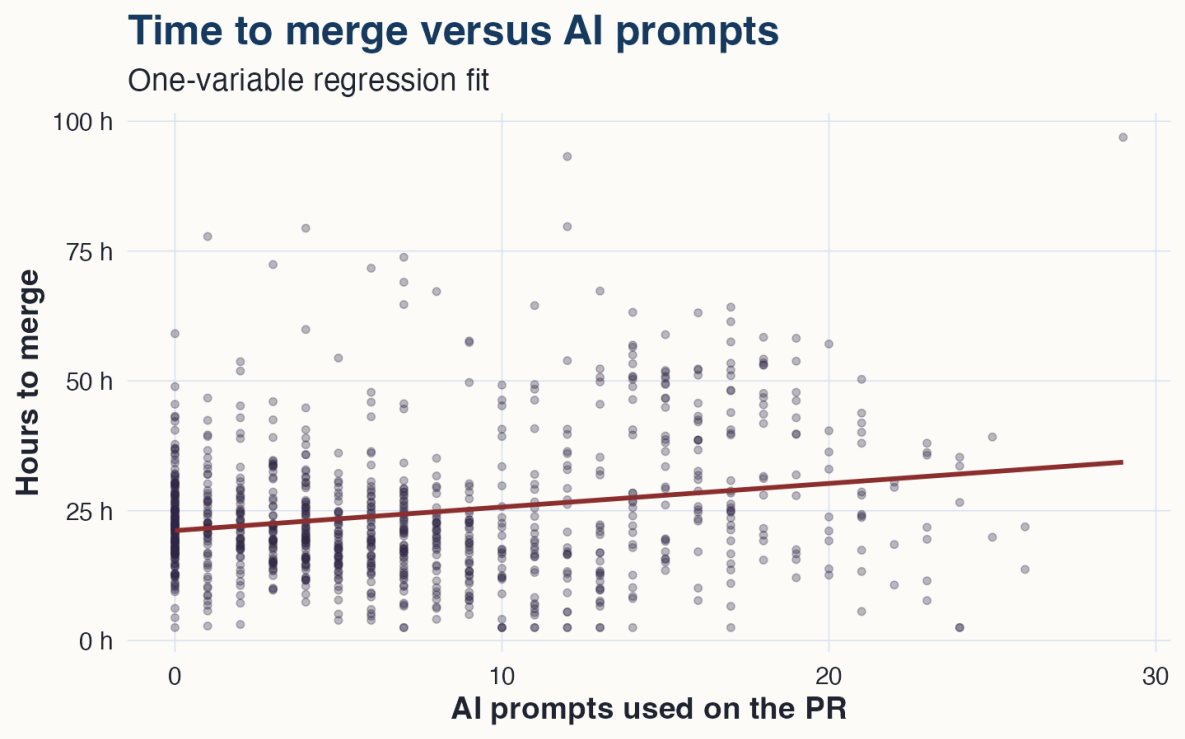
$$\text{HoursToMerge}_i = \beta_0 + \beta_1 \text{AIPrompts}_i + e_i.$$

In the slides, the fitted slope is positive. Pull requests with more AI prompts took longer to merge.

```
ai_naive <- lm(HoursToMerge ~ AIPrompts, data = prs)

tidy(ai_naive)
```

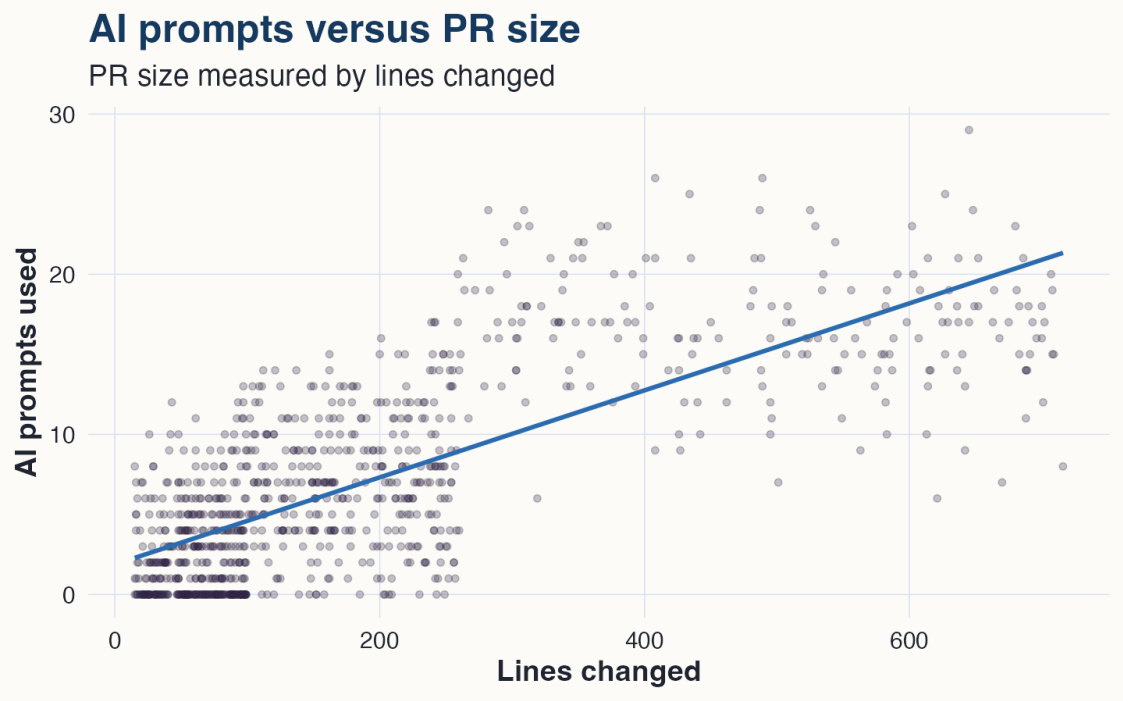
```
# A tibble: 2 x 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)   21.2      0.634      33.4  2.89e-162
2 AIPrompts      0.454    0.0665      6.83 1.54e- 11
```

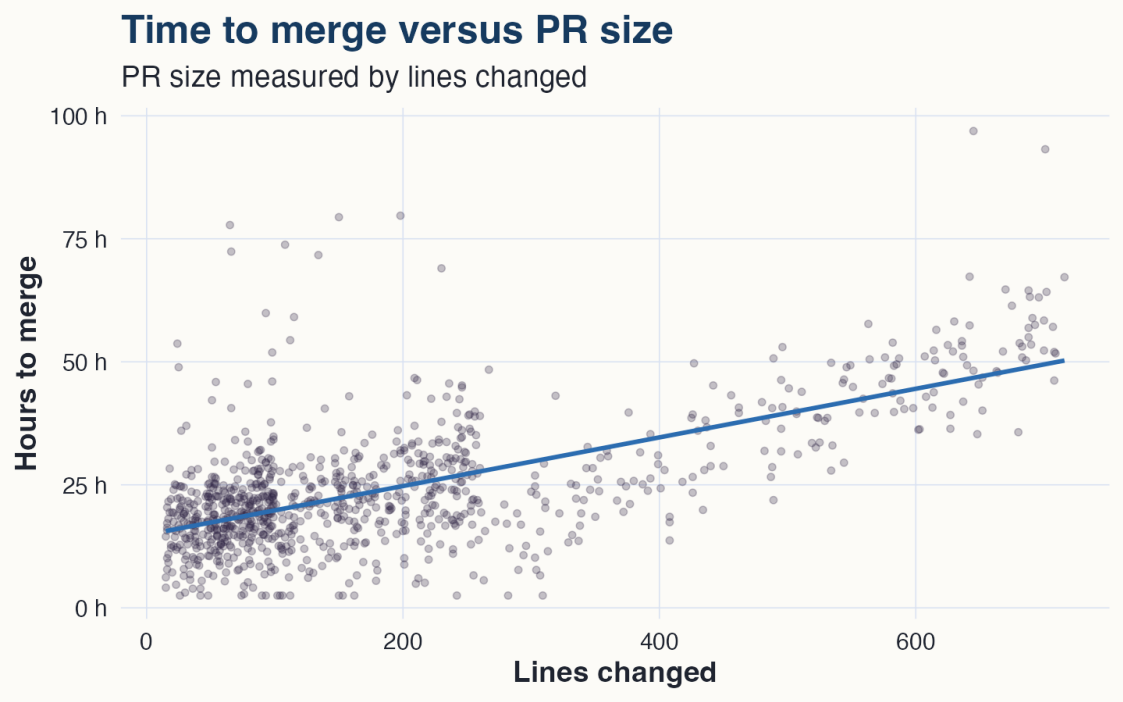


That is not yet a good answer to the question "did AI help?" Larger pull requests use more AI prompts, and larger pull requests take longer to review. The naive slope compares high-prompt pull requests to low-prompt pull requests, but it also compares large pull requests to small pull requests.

That is the wrong comparison.

The two confounding pictures are:





Adjusting for pull request size

Define

$$\text{Lines100}_i = \frac{\text{LinesChanged}_i}{100}.$$

Fit

$$\text{HoursToMerge}_i = \beta_0 + \beta_1 \text{AIPrompts}_i + \beta_2 \text{Lines100}_i + e_i.$$

```
ai_adjusted <- lm(HoursToMerge ~ AIPrompts + Lines100, data = prs)

tidy(ai_adjusted)
```

```
# A tibble: 3 x 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)    17.3     0.427       40.6 9.08e-210
2 AIPrompts      -1.31    0.0657     -20.0 1.46e- 74
3 Lines100        8.52    0.238       35.8 2.05e-178
```

The fitted equation from lecture is

$$\widehat{\text{HoursToMerge}} = 17.3 - 1.3\text{AIPrompts} + 8.5\text{Lines100}.$$

The AI prompt coefficient is now negative. Its interpretation is:

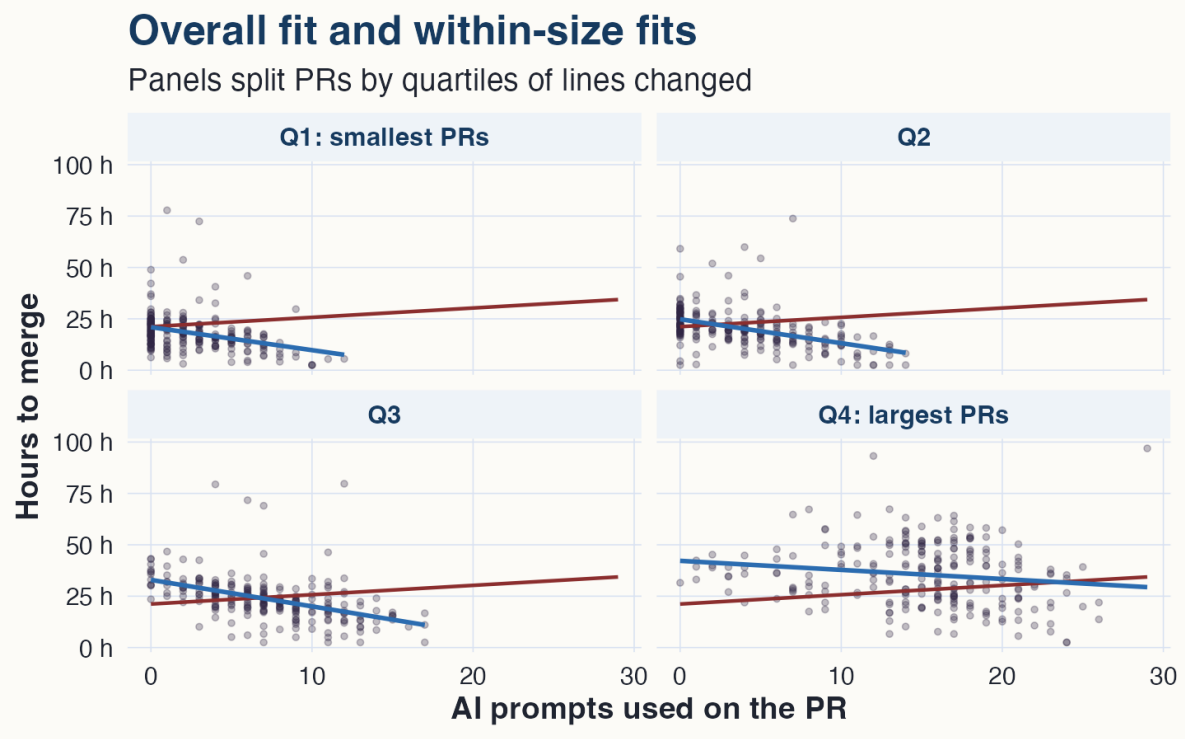
> among pull requests with similar size, one additional AI prompt predicts about 1.3 fewer hours to merge.

The line-size coefficient is:

> among pull requests with similar prompt counts, 100 more changed lines predicts about 8.5 more hours to merge.

Those are partial relationships.

The quartile plot gives a visual preview of the adjustment:



Adjustment as subtraction

Start with the fitted equation:

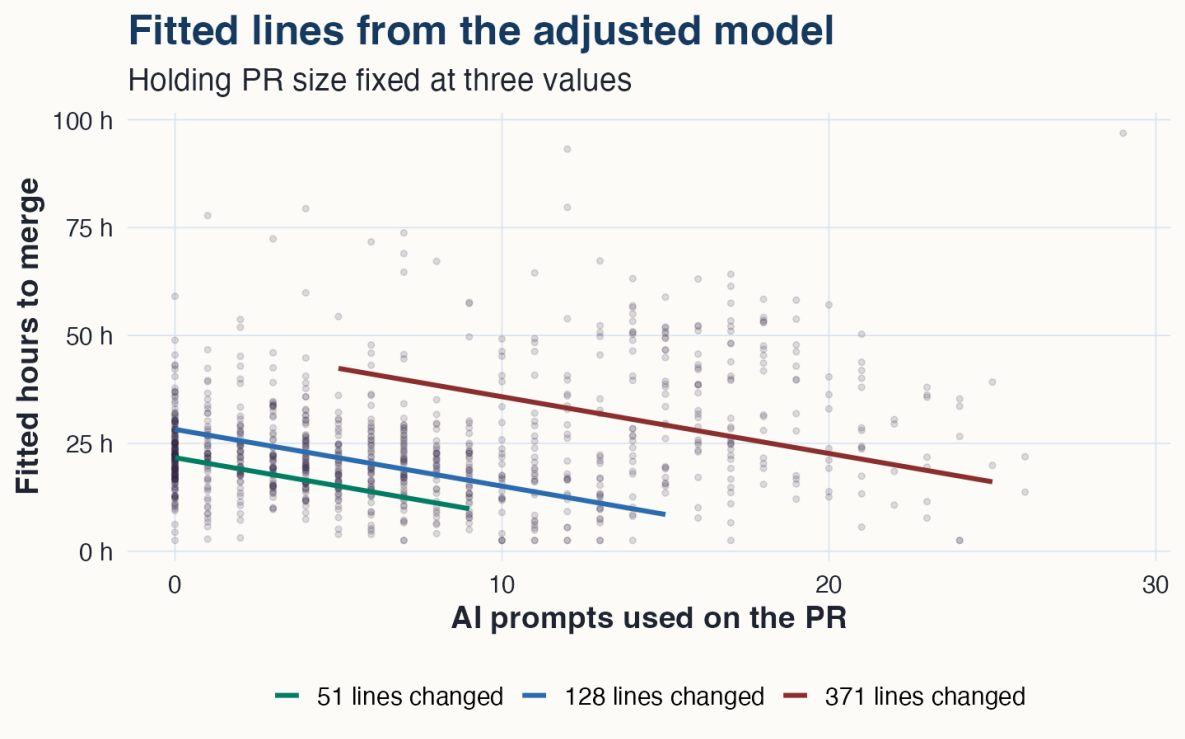
$$\widehat{\text{HoursToMerge}} = 17.3 - 1.3\text{AIPrompts} + 8.5\text{Lines100}.$$

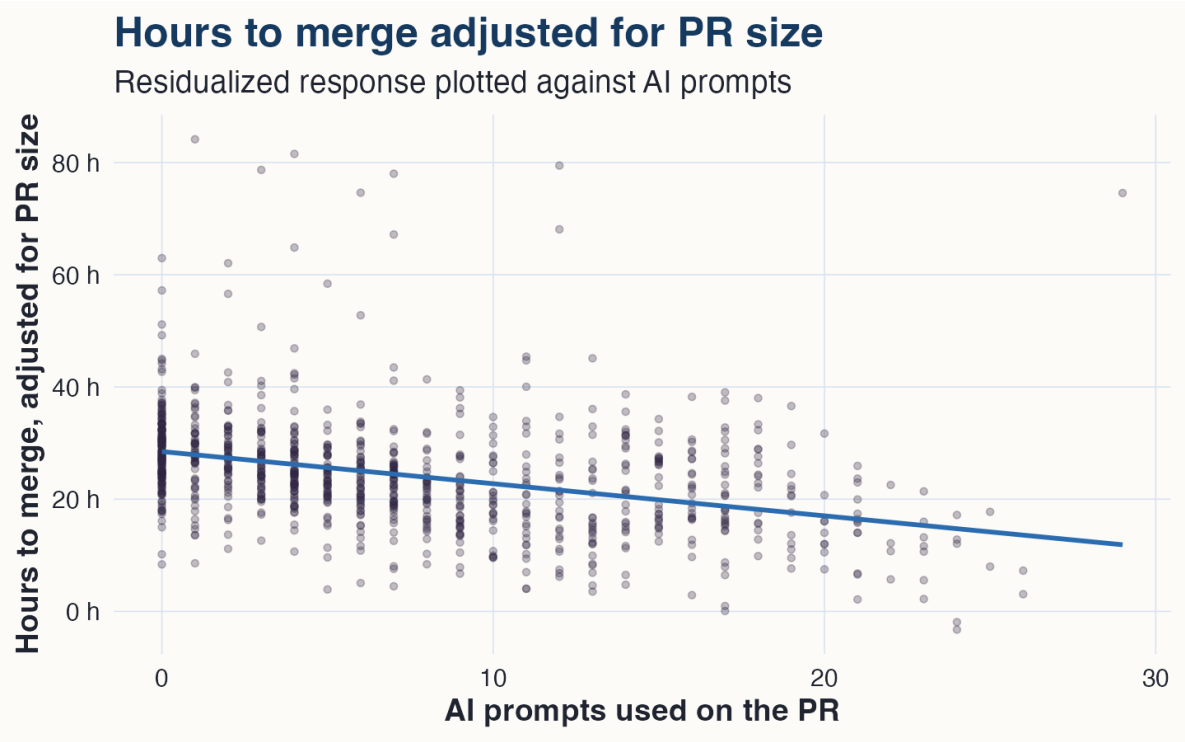
Move the size part to the left:

$$\widehat{\text{HoursToMerge}} - 8.5\text{Lines100} = 17.3 - 1.3\text{AIPrompts}.$$

The left side is merge time adjusted for pull request size. The right side is the partial relationship with AI prompts.

This is the algebra behind the phrase "holding size fixed." Regression removes the part of the outcome associated with size, then reads the prompt relationship from what remains.





Overall versus partial

The overall slope answers:

How do prompt count and merge time vary together in the full cloud of pull requests?

The partial slope answers:

How do prompt count and merge time vary together among pull requests of similar size?

Both are real summaries of the data. They answer different questions.

When predictors are correlated, the overall relationship can point in the opposite direction from the partial relationship. That is not a contradiction. It is a warning about the comparison being made.

A pricing question

The H-E-B meal-kit example asks:

If price rises, how much demand do we lose?

The response is units sold. The main predictor is price. Both are logged, so the price coefficient is a demand elasticity.

The app also records `EligibleViews`: the number of meal-kit page views eligible for the marketing push in that market-week. The sequence matters:

1. a shopper reaches the meal-kit page,
2. conditional on that view, some eligible shoppers get the push,
3. some shoppers buy.

So eligible views are exposure. A week with more eligible views has more chances to sell meal kits.

Read the meal-kit data:

```
meal_kits <- read_csv(file.path(module_dir, "meal_kit_demand.csv")) |>
  mutate(
    Marketing = factor(Marketing, levels = c("No push", "Push")),
    logPrice = log(Price),
    logEligibleViews = log(EligibleViews),
    logUnits = log(UnitsSold),
    logPriceCentered = logPrice - log(23)
  )
```

The naive log-log model is

$$\log(\text{UnitsSold}_i) = \beta_0 + \beta_1 \log(\text{Price}_i) + e_i.$$

In the slides, the naive elasticity is about

$$\hat{\beta}_1 = 1.94.$$

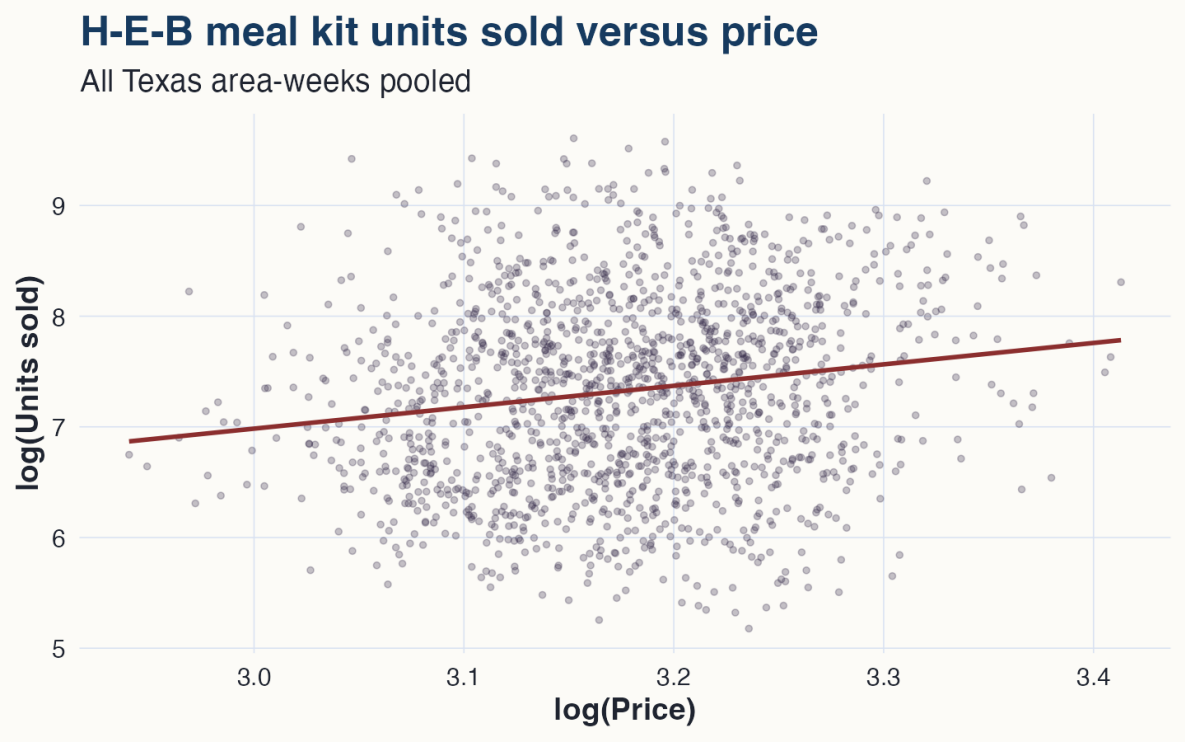
That says higher prices predict more sales. That is not a demand curve. It is a warning that price is tangled up with other variables.

But the app data were not a clean pricing experiment.

```
meal_naive <- lm(logUnits ~ logPrice, data = meal_kits)

tidy(meal_naive)
```

```
# A tibble: 2 x 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)     1.17     0.944      1.24 2.14e- 1
2 logPrice        1.94     0.297      6.51 9.85e-11
```



Marketing confounding

The app sometimes runs a homepage or push promotion. Those marketing pushes increase demand. They also appear more often in cheaper weeks.

So price is tangled up with marketing:

$$\log(\text{UnitsSold}) \sim \log(\text{Price}) + \text{Marketing}.$$

After adding the marketing indicator, the price slope is still positive:

2.09.

It is still the wrong sign. Marketing adjustment alone has not fixed the comparison.

```
meal_kits |>
  group_by(Marketing) |>
  summarize(
    weeks = n(),
    average_price = mean(Price),
    average_eligible_views = mean(EligibleViews),
    average_units = mean(UnitsSold),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 5
  Marketing weeks average_price average_eligible_views average_units
  <fct>     <int>         <dbl>                   <dbl>         <dbl>
1 No push     879          24.2                  17368.         1883.
2 Push        681          23.9                  16093.         2615.
```

```
meal_marketing <- lm(logUnits ~ logPrice + Marketing, data = meal_kits)

tidy(meal_marketing)
```

```
# A tibble: 3 x 5
  term          estimate std.error statistic  p.value
  <chr>            <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)      0.548    0.938      0.585 5.59e- 1
2 logPrice         2.10     0.295      7.11  1.72e-12
3 MarketingPush    0.276    0.0433     6.38  2.31e-10
```

Exposure adjustment

Now add the second numerical predictor:

$$\log(\text{UnitsSold}) \sim \log(\text{Price}) + \log(\text{EligibleViews}) + \text{Marketing}.$$

The fitted slopes are about

$$\hat{\beta}_{\log(\text{Price})} = -2.71, \qquad \hat{\beta}_{\log(\text{EligibleViews})} = 1.81.$$

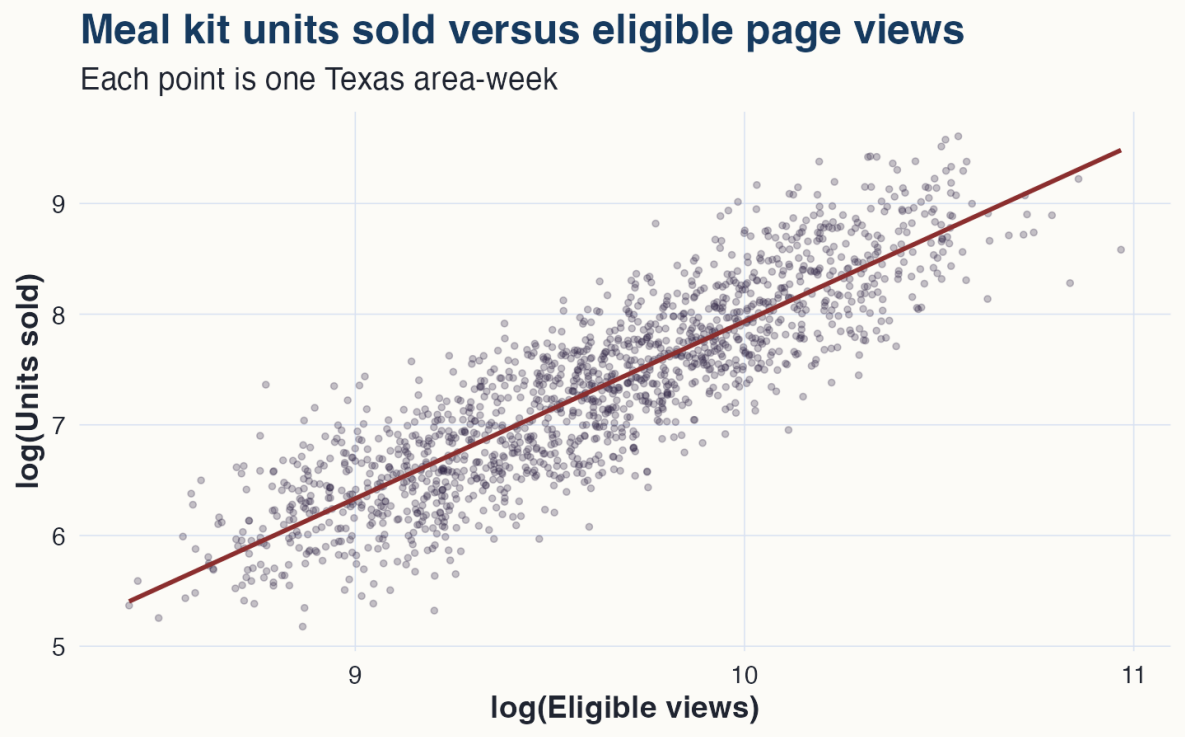
The price slope is now negative. Eligible views are doing what they should: more exposed shoppers means more expected sales.

But market differences are still mixed into the comparison.

```r
meal_exposure <- lm(logUnits ~ logPrice + logEligibleViews + Marketing, data = meal_kits)

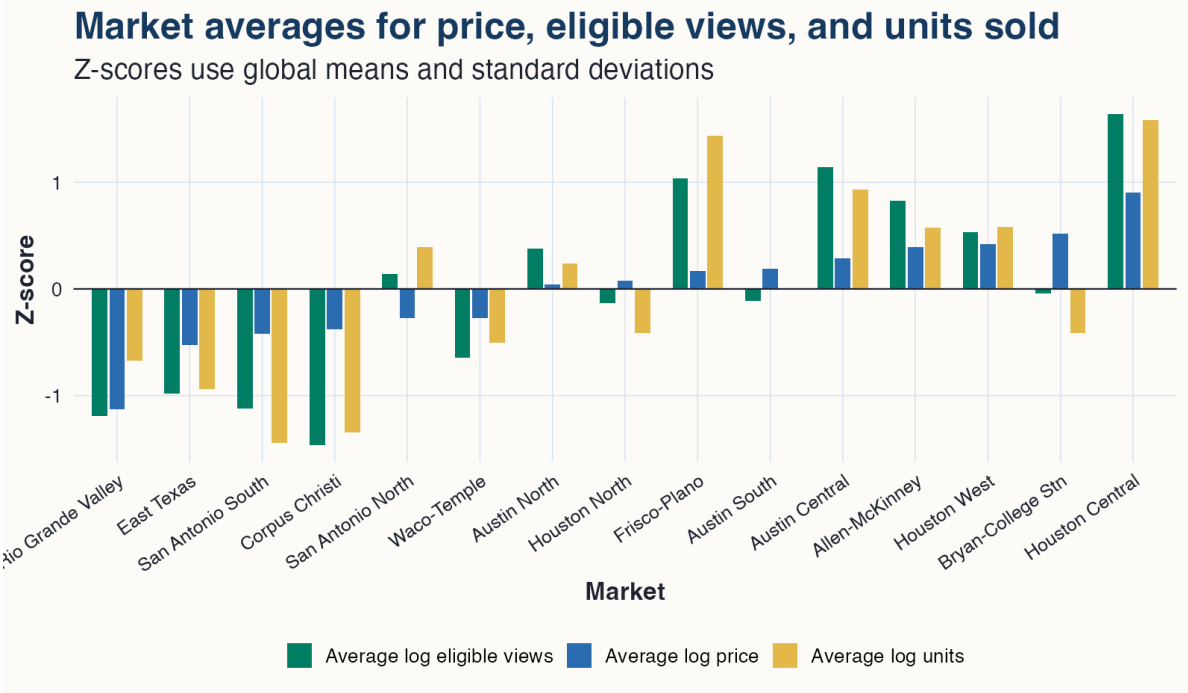
tidy(meal_exposure)
```

```
# A tibble: 4 x 5
  term              estimate std.error statistic  p.value
  <chr>                <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)        -1.66     0.365      -4.55 5.85e- 6
2 logPrice           -2.71     0.125     -21.6  9.41e-91
3 logEligibleViews    1.81     0.0193     93.7  0
4 MarketingPush       0.380    0.0168     22.5  1.44e-97
```



Market adjustment

H-E-B areas differ in baseline demand and price level. A high-price area can also be a high-demand area. If we pool all areas, the regression partly compares one Texas area to another.



Add area dummies:

$$\log(\text{UnitsSold}_i) = \beta_0 + \beta_1 \log(\text{Price}_i) + \beta_2 \log(\text{EligibleViews}_i) + \beta_3 \text{Push}_i + \gamma_{\text{Market}_i} + e_i.$$

Now the price coefficient compares weeks in the same area, with similar eligible page views and the same marketing status.

The fitted elasticity is about

$$\hat{\beta}_1 = -2.35.$$

That is a different business answer. A 1% price increase predicts about 2.35% fewer units sold, after adjusting for eligible views, marketing, and market.

```
meal_market <- lm(
  logUnits ~ logPrice + logEligibleViews + Marketing + Market,
  data = meal_kits
)

tidy(meal_market) |>
  filter(term %in% c("logPrice", "logEligibleViews", "MarketingPush"))
```

```
# A tibble: 3 x 5
  term             estimate std.error statistic   p.value
  <chr>               <dbl>     <dbl>     <dbl>     <dbl>
1 logPrice           -2.35     0.0822     -28.6 2.78e-144
2 logEligibleViews    0.995    0.0266      37.3 7.18e-218
3 MarketingPush       0.427    0.0109      39.1 8.57e-233
```

Revenue implication

For a log-log demand model,

$$\text{revenue elasticity} \approx 1 + \text{demand elasticity}.$$

With demand elasticity -2.35:

$$1 + (-2.35) = -1.35.$$

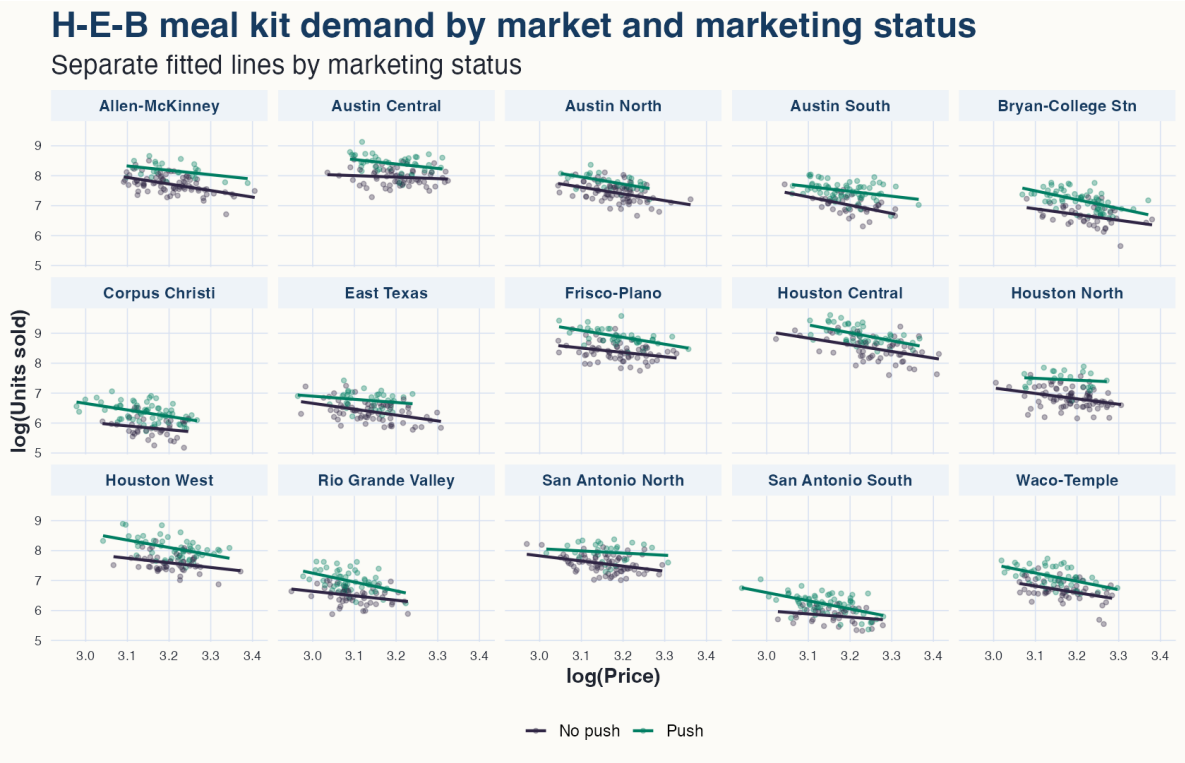
So a 1% price increase predicts roughly:

- 2.35% fewer units sold,
- 1.35% lower revenue.

The naive model made the product look safer to mark up than it probably is.

Interactions

Marketing can also change the price slope. A push might bring in more price-sensitive customers. Then the demand elasticity during push weeks might differ from the elasticity during no-push weeks.



Use an interaction:

$$\log(\text{UnitsSold}_i) = \beta_0 + \beta_1 x_i + \beta_2 \text{Push}_i + \beta_3 x_i \text{Push}_i + \beta_4 \log(\text{EligibleViews}_i) + \gamma_{\text{Market}_i} + e_i,$$

where

$$x_i = \log(\text{Price}_i) - \log(23).$$

The centering means $x_i = 0$ at a price of about \$23. That gives the push coefficient a clear interpretation:

> the fitted push-week lift at about \$23, after adjusting for market.

Reading the interaction

From the lecture:

$$\hat{\beta}_1 = -2.23, \qquad \hat{\beta}_3 = -0.28.$$

So the no-push elasticity is

$$-2.23.$$

The push-week elasticity is

$$-2.23 + (-0.28) = -2.51.$$

The interaction coefficient is the extra price slope in push weeks. Here it says push weeks are a bit more price-sensitive.

The push dummy coefficient is about 0.44 on the log scale. At about \$23, that corresponds to

$$e^{0.44} - 1 \approx 0.55,$$

or about 55% higher expected unit sales, after adjusting for eligible views and market.

```
meal_interaction <- lm(
  logUnits ~ logPriceCentered * Marketing + logEligibleViews + Market,
  data = meal_kits
)

tidy(meal_interaction) |>
  filter(term %in% c(
    "logPriceCentered",
    "MarketingPush",
```

```
    "logEligibleViews",
    "logPriceCentered:MarketingPush"
  ))
```

```
# A tibble: 4 x 5
  term                           estimate std.error statistic   p.value
  <chr>                             <dbl>     <dbl>     <dbl>     <dbl>
1 logPriceCentered                  -2.23     0.103     -21.6  6.96e- 91
2 MarketingPush                      0.438    0.0123     35.5  2.53e-202
3 logEligibleViews                   0.995    0.0266     37.4  2.64e-218
4 logPriceCentered:MarketingPush    -0.284    0.144      -1.97 4.88e-  2
```

```
no_push_elasticity <- coef(meal_interaction)[["logPriceCentered"]]
push_elasticity <- no_push_elasticity +
  coef(meal_interaction)[["logPriceCentered:MarketingPush"]]

tibble(
  marketing_status = c("No push", "Push"),
  estimated_elasticity = c(no_push_elasticity, push_elasticity)
)
```

```
# A tibble: 2 x 2
  marketing_status estimated_elasticity
  <chr>                           <dbl>
1 No push                         -2.23
2 Push                            -2.51
```

Fixed effects as many dummies

The market terms

$$\gamma_{\text{Market}_i}$$

are just dummy-variable offsets. In applied work, people often call these fixed effects.

A market fixed effect gives each area its own baseline. It does not give each area its own price slope unless we add interactions with market.

So:

$$\log(\text{UnitsSold}) \sim \log(\text{Price}) + \text{Market}$$

means one common price slope with many market baselines.

By contrast,

$$\log(\text{UnitsSold}) \sim \log(\text{Price}) * \text{Market}$$

allows the price slope to differ by market.

What to carry forward

Multiple regression is not magic. It changes the comparison.

The question is always:

> What units are being compared by this coefficient?

For the AI pull request case, the useful comparison is prompt use among pull requests of similar size.

For the H-E-B meal-kit case, the useful comparison is higher-price weeks versus lower-price weeks in the same Texas area, with similar eligible page views and the same marketing condition.

A partial slope is only useful when the adjusted comparison matches the question you meant to ask.